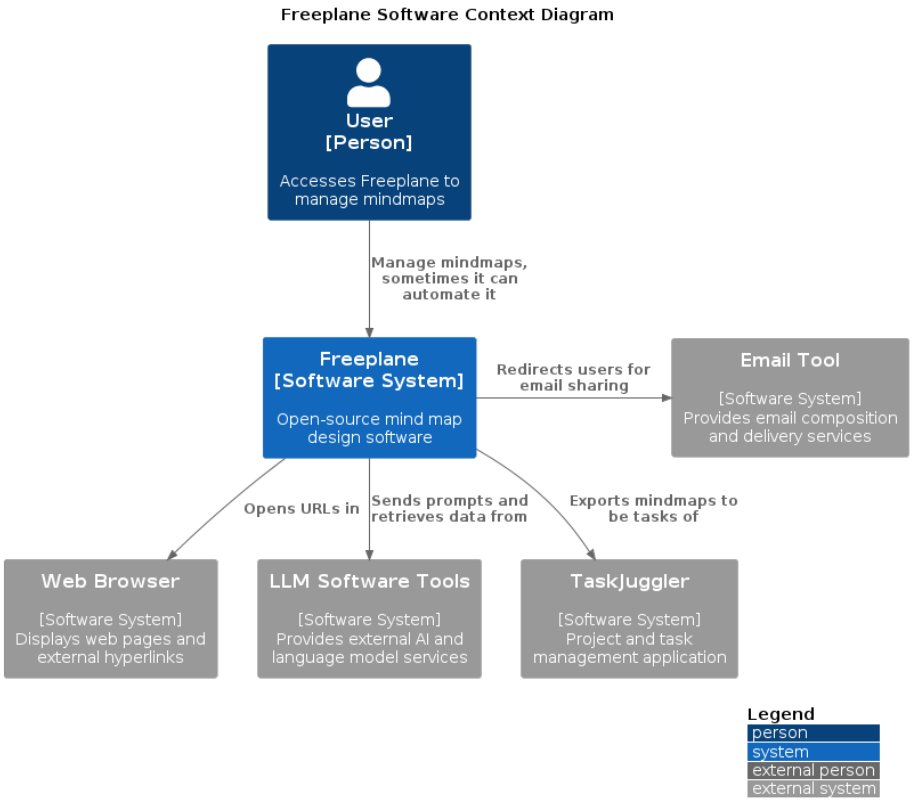
Architectural Analysis of Freeplane – Reverse Engineering and Evaluation

1. Introduction and Analysis Methodology This report presents an architectural analysis of the Freeplane mind-mapping software, reverse-engineered via static code analysis, documentation review, and repository statistics. Architecturally, it is an imperfect micro-kernel monolith based on the OSGi framework. The core module implements business logic, while plugins extend functionalities. This design breaks strict micro-kernel principles, hence the imperfect definition.

2. The System in its Ecosystem: C4 Context Model Freeplane was born as a fork of the well-known Freemind software. The official documentation reports that the decision was taken to improve software's design and to speed up its development and maintenance cycles.



Although being represented as a single person, users can be distinguished in two groups: basic users, that exploit the software for basic mindmapping tools, and advanced, computer literate users that can write custom scripts to enhance Freeplane features. These groups have been merged in a single Actor since

they share the same GUI and potentially all basic users can also use advanced features.

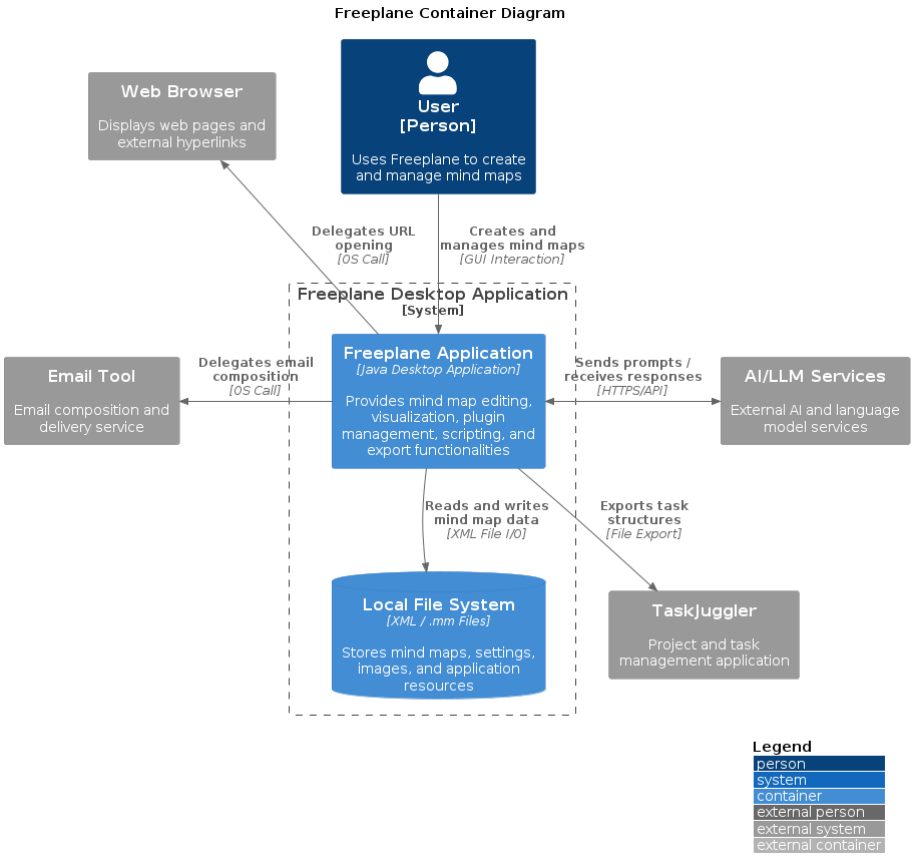
The persistence layer is managed internally by the software: users can define their preferred path and the software interacts with the Operating System to save it with the desired format.

Two types of interaction with external systems can be identified:

- Task delegation: the system depends on external tools that are responsible for handling operations outside its own scope. The relationships with the Browser or with the Email tool fall into this category — clicking a URL or clicking an email address triggers actions that Freeplane delegates entirely to the appropriate external system.
- Data delegation: the dependency involves an active exchange of information. The integration with TaskJuggler requires Freeplane to transform and adapt the content of a mind map into a structured set of tasks before transferring it to the external system. This is a unidirectional relationship
  The interaction with LLM systems also falls into this category, but the relationship is bidirectional: sending a prompt transfers data to the LLM API, which in turn sends information back through its response.

The tension between different kinds of dependencies reveals that the software has grown over the years to support different features to meet requirements that can be very different among users.

3. Decomposition and Runtime: C4 Container Model The Container Model aims at showing how the software is built, from a lower, more detailed standpoint.



Freeplane is represented as a single Container software. This choice can be explained by the nature of its technology stack. Freeplane has multiple plugins, that are connected to the core through the OSGi framework. This implementation ensures separation among plugins, making them independent from one another; theoretically, Freeplane plugins can be autonomously developed, tested and integrated in the environment. Moreover, they do not extend core software functionalities: plugins bring advanced features, both graphical and functional. The core software alone would work without plugins.
However, the software has been represented as a single Component unit because OSGi bundles have not independent life. Even though they are developed externally, they require the OSGi engine to be executed. Plugins such as `freeplane script` or `freeplane latex` cannot exist outside the Freeplane environment. That is because all plugins plugins are designed to extend features in the core Freeplane application. Furthermore, they cannot be even run outside the launcher defined in the core `freeplane` package: the OSGi implementation require the framework to start plugins with a sequential process, managed by a kernel that is instantiated in the launcher implemented in the core package.

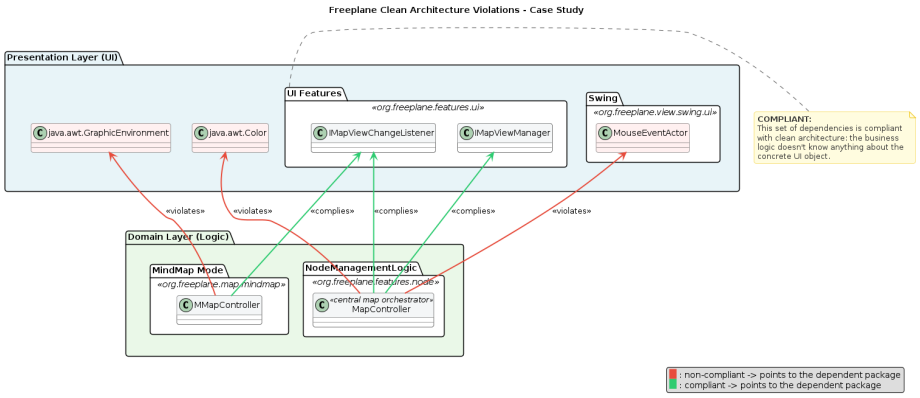
These consideration have led the analysis to consider the software as composed of a single container. In this situation, independent deployability cannot justify the definition of plugins as independent units.

The persistence layer is managed with the Operating System. Freeplane stores information related to the single mind-map in a proprietary file format, that is the `.mm`. This is an XML-derived format that fits the definition of mindmaps as nested blocks of nodes. Users can independentely decide which area of their File System save data to.

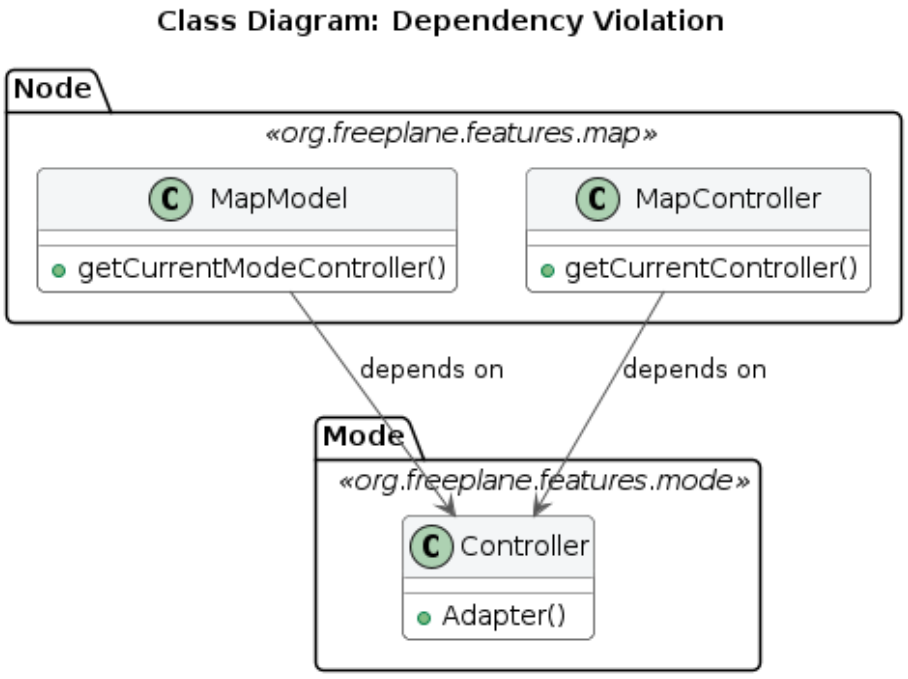
4. Mapping to Clean Architecture: Theory vs. Reality (Target: ~500 words) This section aims to clarify the architectural choices made to build the software, and to define if Clean Architecture principles are respected. The first step is to define *entities*, *use cases* and *external layers*, and where to find them. The package `org.freeplane.features.map` is the perfect candidate: its classes define core software logic, such as how nodes are defined and organized within the map.

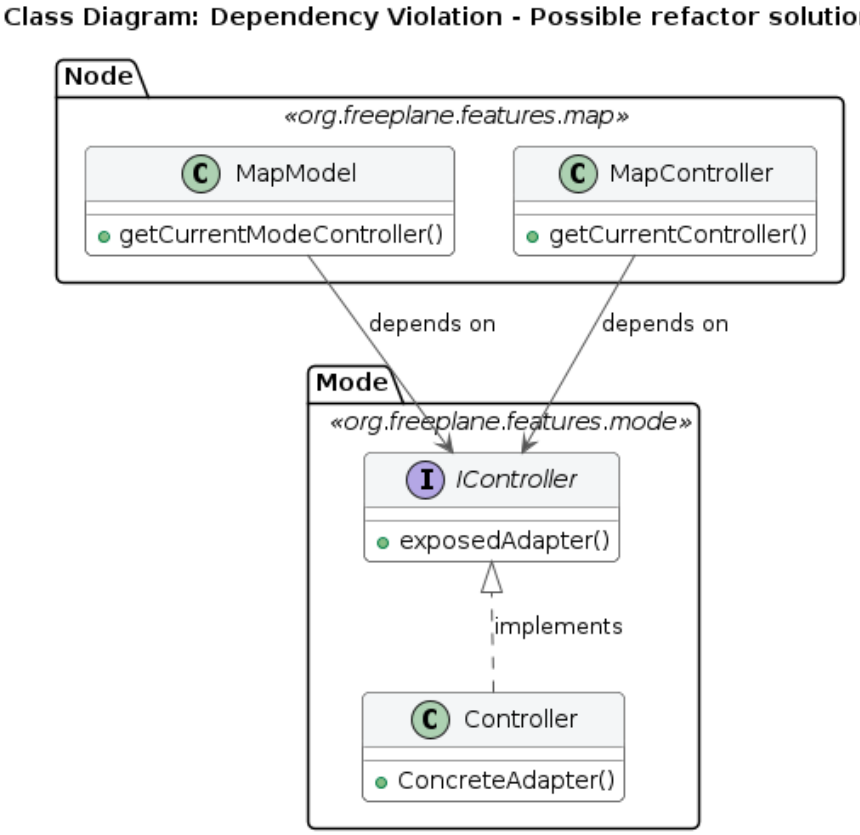
To double check, data from the official repository was analyzed. Particularly, we tried to estimate package stability from the number of commits directly involving the package. The first result seemed to contradict the thesis: `org.freeplane.features.map` is a very unstable package, with hundreds of commits during the software lifecycle. This can be explained by the high coupling between the package and UI components. A deeper analysis reveals that almost 43% of commits share changes with freeplane UI components, and this number grows if we look at subpackages such as `org.freeplane.features.map.filemode` or `org.freeplane.features.map.clipboard` (both at almost 61% of shared commit number with ui components). This data suggests that the Clean Architecture patterns are not well-respected in the software: core business logic should have almost no co-change with UI and graphical features.

Code analysis reveals that most classes in the package have dependencies on the frontend, involving both custom Freeplane UI classes and standard Java AWT ones. There is a mixed approach: in many cases classes import Interfaces from the `org.freeplane.features.ui` package, that represent the abstraction for the User Interface management. However, sometimes there is direct interaction with frontend classes: for instance, class `MapController` manages map view through the `IMapViewManager` interface, but it implements a concrete method for managing an external peripheral (the mouse, through `MouseEventActor`). Many concrete standard Java UI classes are imported as well. This mixed approach results in high-coupling between the business logic and external layers in the architecture, making it less isolated and more difficult to be tested and extended or modified.



There are other crucial violations of the Clean Architecture pattern: there is no clear definition of *entities*, *use cases* and other layers. Some classes may look similar to one of these concepts: `MapModel` could be classified as an *entity*, `MapController` as a *use case*, `Controller` from `org.freeplane.features.mode` as an *adapter*. Most critical violations of the Clean Architecture pattern can be found in the dependency among these three classes: `MapModel` and `MapController` depend on the `Controller`; the flow of dependencies is broken and therefore inner business logic cannot be tested in isolation.





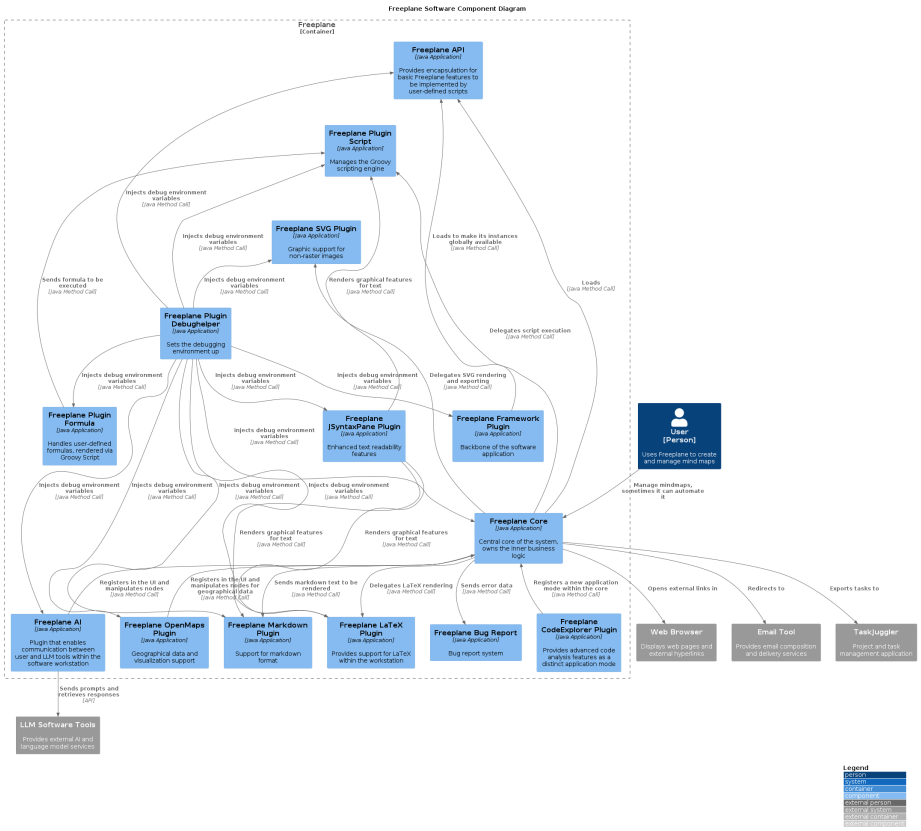
This situation leverages the Dependency Inversion Principle to decouple the three classes, and it introduces a Boundary Interface that exposes signatures which outer components must adhere to. This way, the set of involved classes is compliant with the Clean Architecture pattern.

Compliance with the principles from the Clean Architecture pattern can be found in the *persistence layer*: classes such as `MapReader` and `MapWriter` are at the outer layer of the architecture, and there are no dependency violations. They perform operations to save or read data from the mindmap directly in the `.mm` file. The `org.freeplane.features.filter` package suffers from the same set of problems: its `FilterController` has the same mixed approach at User Interface import dependencies, and it mixes business logic, application logic, and frontend concerns in the same class.. That's why architectural flaws from `org.freeplane.features.map` package can be fairly extended to the whole software structure.

To sum up, the software does not fully comply with Clean Architecture principles. There are many violations that mainly concern architectural boundaries: the most serious violation is the lack of clear division between entities and use cases,

and the broken dependency flow. Dependency on concrete User Interface methods makes the software more difficult to be tested in isolation. These violations have an explanation: as reported in the Official Documentation, core classes were designed to be extensible, and to follow the Extension-Object Design Pattern defined by Erich Gamma. This architectural choice aims at building extensions to well-defined objects. The lack of compliance with the Clean Architecture can be explained by the need to have both entities and use cases in the same set of classes to make them easily extensible. However, Extensible Object theory does not justify the direct implementation of UI elements in core entities. This remains a pure violation of the Separation of Concerns at an architectural level.

5. Zooming into the Engine: C4 Component Model of the Core (Target: ~600 words) The Component Model offers the deepest view of Freeplane's internal structure, detailing how the single container introduced in Section 3 decomposes into individually deployable OSGi bundles and the external libraries they depend upon.



The diagram reveals a layered hierarchy that governs how the system bootstraps and how data flows between components.

Framework Plugin. The Freeplane Framework Plugin sits at the top of the dependency tree. It initializes the OSGi runtime environment and loads the API plugin, making its instances globally available to every downstream bundle. It acts as the entry point through which the entire component graph is wired together.

Freeplane Core. Directly below the framework resides the Freeplane Core, the central component that owns the inner business logic: map models, node structures, mode controllers, and event dispatching. The Core loads the API and delegates specialist work downward — script execution, markdown and LaTeX rendering, SVG export, and error reporting — to the appropriate plugins via standard Java method calls.

This is where the MVC triad lives: `MapModel` and `NodeModel` define the domain entities, while `MapController` and `ModeController` orchestrate user actions and propagate model changes to the UI.

However, as discussed in Section 4, this orchestration layer suffers from significant design flaws. Controllers such as `MapController` are *God Objects* that aggregate I/O setup, action registration, navigation, folding, and event orchestration in a single class — a systemic **Single Responsibility Principle (SRP)** violation.

The same pattern recurs in `FilterController` (1,179 lines) and `ModeController` (491 lines, 10+ distinct responsibilities).

Furthermore, the pervasive use of `Controller.getCurrentController()` — a concrete global singleton called hundreds of times across the `features` layer — constitutes the most widespread **Dependency Inversion Principle (DIP)** violation in the codebase.

`MapWriter` similarly instantiates concrete dependencies rather than receiving abstractions.

These patterns couple nearly every component to concrete implementations rather than abstractions.

API and Lateral Plugins. The Freeplane API provides an encapsulation layer that exposes core features for user-defined Groovy scripts, shielding script authors from internal implementation details. Sitting alongside the API are four lateral plugins: the AI plugin (LLM integration via LangChain4j), OpenMaps (geographical visualization via JMapViewer), CodeExplorer (architectural analysis via ArchUnit and JGraphT), and the Bug Report module.

An important architectural observation emerges here: while the Core pushes data downward to these plugins, three of them — AI, OpenMaps, and CodeExplorer — also call *upward* into the Core to register UI elements and manipulate nodes. This bidirectional dependency is managed at runtime through the OSGi service layer, but it reveals tension in the component hierarchy.

Notably, `NodeLevelConditionController.createASelectableCondition()` uses `if`-chains to select condition types — adding a new condition requires

modifying existing code, which violates the **Open/Closed Principle (OCP)**.

In contrast, the `filter.condition` subpackage demonstrates OCP *compliance* through a Strategy/Decorator pattern (`ASelectableCondition`, `DecoratedCondition`), showing that developers have applied the principle inconsistently.

Script Engine. The Plugin Script component manages the Groovy scripting engine and resolves script dependencies via Apache Ivy. It serves as the execution substrate for the script-dependent plugins described below.

Script-Dependent Plugins. Formula, Markdown, LaTeX, and JSyntaxPane all require the scripting infrastructure. Formula sends expressions to the script engine for evaluation; Markdown and LaTeX delegate to external rendering libraries (Markdj and JLatexMath respectively); JSyntaxPane enhances text readability and cross-cuts across Script, Markdown, and LaTeX components.

A **Liskov Substitution Principle (LSP)** concern surfaces at this level: `SingleCopySource` extends `NodeModel` but throws `RuntimeException` for inherited methods, breaking the substitutability contract.

Similarly, `IMapSelection` bundles selection, navigation, scrolling, filtering, and visibility into a single fat interface — an **Interface Segregation Principle (ISP)** violation — whereas `INodeChangeListener`, with its single `nodeChanged` method, represents a clean, minimal counterexample.

SVG Plugin. The SVG component handles non-raster image rendering via Apache Batik and PDF transcoding via Apache FOP.

Cross-Cutting: Debug Helper. The Debug Helper plugin stands apart from the main hierarchy: it injects debug environment variables into every other component, from the Framework down to OpenMaps. This cross-cutting nature means it touches all layers without belonging to any single one.

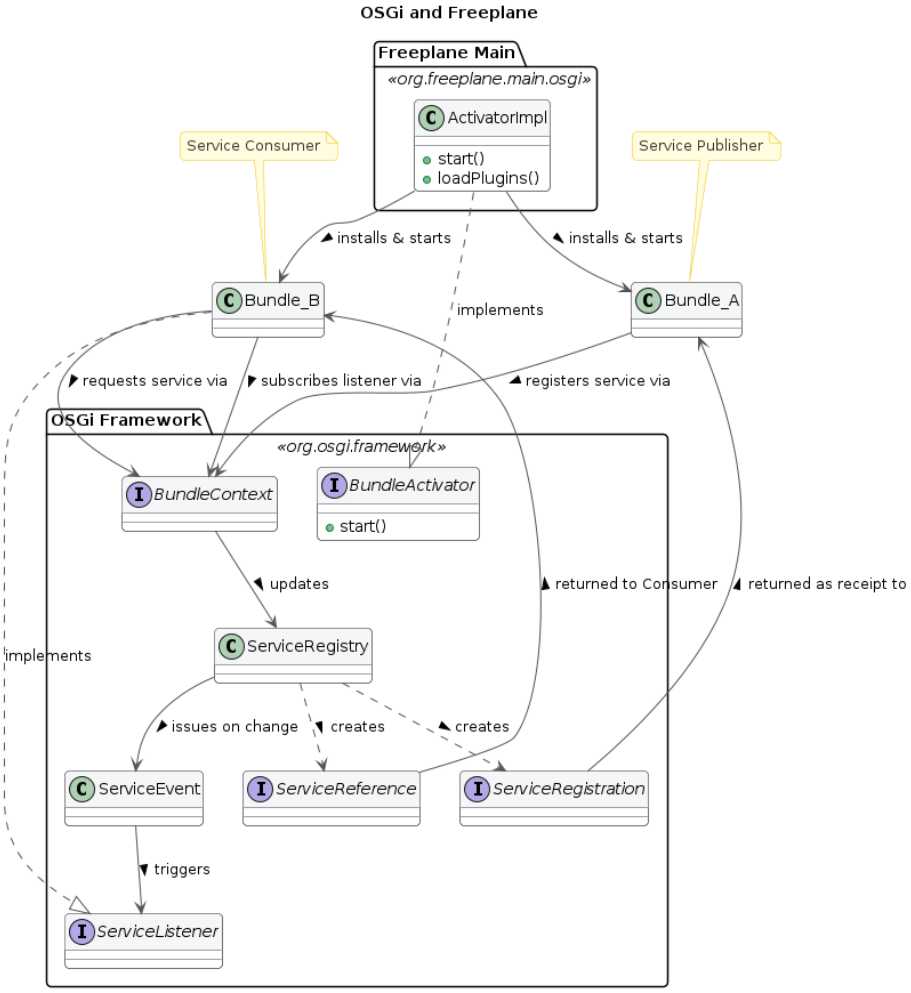
The SOLID violations catalogued above are not isolated incidents confined to `org.freeplane.features.map`. They are **systemic architectural patterns** that repeat across the entire Core, reinforcing the assessment from Section 4 that business logic, application logic, and UI concerns are insufficiently separated throughout the codebase.

Boundaries: Core-Plugin interaction and Internal Business Logic Boundaries analysis

Core-Plugin Interaction Freeplane is built on top of the OSGi framework, in the Knopflerfish implementation. This technology stack allowed developers to build a modular software, where different plugins can be attached to the core easily. The framework generates a three-layer hierarchy: 1. Module Layer: it is the declarative layer: it defines bundles identity and static dependencies. In Freeplane, the `org.freeplane.core` package plays this role. This piece of information can be easily undestood by reading its `MANIFEST.MF` file when compiled:

it stores all settings and dependencies for building the application. 2. Activator: it is responsible for building the infrastructure to make plugins communicate. In Freeplane, it is the ActivatorImpl class in `org.freeplane.main.osgi`. It can be identified since it implements the `org.osgi.framework.BundleActivator` class. 3. Service Layer: bundles provide services, that are registered within the Framework exploiting mechanisms that are built in OSGi environment. Such services can be published or unpublished at anytime. They look like plain Java Objects.

The OSGi bundles are built to be independent from one another: there is no shared knowledge, code or data, and all references to classes or methods in other plugins throw OSGi errors. Plugins and Core have therefore strict boundaries, that can be crossed by pointing to published services only.
At the Module Layer, circular dependencies are critical and they can lead to deadlock: it can happen when thread 1 starts Bundle 1 which requires a Service from Bundle 2, and it stops until it is published. Thread 2 starts Bundle 2 which requires a service from Bundle 1. It stops. The application is locked permanently. This can be solved by implementing additional features provided by the framework.
Freeplane resolves these dependencies at runtime, since it provides a single Activator that is responsible of building the application attaching different plugins. On the other hand, on the Service Layer, a clear dependency tree cannot be drawn. That is because Bundles can both publish and exploit services published by other Bundles, that will likely require Service published by plugins that exploit their services. Dependencies are therefore resolved at run-time: bundles may or may not publish or request services. This pattern makes debugging more difficult, because sometimes it is difficult to track flow of information across bundles in Freeplane.



This structure provides both freedom and constraints to developers: new features can be added by creating a new dedicated plugin. Moreover, since bundles are independent and well separated, deployment and maintenance does not affect the overall system. Caveats of this architectural choice are the programming language and the complete compliance to the framework: developers must write all plugins in Java (the OSGi framework doesn't support other programming languages), and they must write plugins that implement the logic described above.

Code Analysis: Boundaries in Freeplane Core The object of this short analysis is to understand how boundaries are respected at the core level. The analysis is carried on the `org.freeplane.features.map` package. This package is representative of the overall architecture, since it includes classes that manage

core software logic, such as Nodes and MindMap models. Data was extracted from the main repository, exploiting `git log`, and analysis is carried out through the logical coupling analysis method, where groups that change together frequently have been put in the same cluster. The co-change threshold (minimun number of git commits classes must share to be included in the analysis) is 10.
Many Common Closure Principle violations can be found: it means that classes often change together, even if they shouldn't. Most of them are related to changes that affect the `MapController` class and many visual components that are related to `Swing` library. This data suggest loose boundaries, at least between entities and UI features. Within the package, classes that host the most important business logic features rarely change together: for instance, `MapModel` and `NodeModel` never change in the same commit. They do not have static dependencies on each other either. That is proof that at business level core components are well separated. That is evidence that the `freeplane.org.features.map` acts as a generic container, where different logic coexists. This is an architectural flaw that can reduce testability and isolation. Even though in clear violation of Clean Architecture principle, a lower, looser level of component separation has been put in place.

7. Architectural Characteristics and Conclusions Extensibility (Strong) Two-level: OSGi plugin isolation (macro) + Extension Object Pattern on nodes (micro). 10 official plugins developed independently. **Maintainability** (Weak) God Object controllers with too many responsibilities. Pervasive global singleton calls create invisible state dependencies.
Modularity (Mixed) Strong between plugins (OSGi, no cross-bundle change correlation). Weak within core: nearly half of core commits co-change with UI components. **Testability** (Weak) No dependency injection. Over half the domain layer depends on GUI frameworks. Global singletons cannot be replaced with test doubles.

Coupling and Cohesion Metrics

Derived from `git log` clusterization (threshold: 10 shared commits):

- **Low entity coupling (positive):** `MapModel` and `NodeModel` never co-change — well-separated domain classes.
- **High domain-UI coupling (negative):** `MapController` frequently co-changes with Swing components.
- **Strong plugin isolation (positive):** OSGi bundles show no cross-bundle change correlation.

Conclusion

Freeplane balances extensibility and legacy constraints. Its OSGi plugin system and Extension Object Pattern provide robust, two-level extensibility. However, the core suffers from systemic architectural debt: God Object controllers, pervasive singleton dependencies, and over half the business logic layer coupled to

AWT/Swing. These issues, rooted in the software's 2009 origins, make the core untestable in isolation and resistant to UI migration.

If an architectural refactoring were to be proposed based on Clean Architecture principles, the priority would be introducing abstraction interfaces for GUI dependencies (`Color`, `Font`, `Component`) in the domain layer, enabling independent testability without breaking existing functionality.